Hardware Backends

Deep Dive into SGLang — Chapter 29

Yin Yang

Foundation Books Project

Where this chapter sits

This is the chapter that turns "run it on another device" into a serving decision the runtime must prove.

- Volume I supplied the `ForwardBatch`: request ids, token positions, KV ownership, and sampling state.
- One trace carries the whole chapter: a `Qwen/Qwen3-8B` step with two decode-resident requests and one short extend suffix.
- Changing the device may change allocators, kernels, streams, graph paths, and tolerances — it must *not* change what the step means.

The problem is portability without semantic drift: one serving interface fed by many device capability surfaces.

Roadmap

The portability contract

Discovery, memory, and execution

Correctness and evidence

The portability contract

One serving contract, many device surfaces

Definition (Backend contract)
For device family d, $B_d = (\mathcal{O}_d, \mathcal{M}_d, \mathcal{F}_d, \mathcal{N}_d)$: operation surface, memory semantics, feature flags, and numerical behavior promised to the runtime *before* requests execute.

- $\mathcal{O}_d$: paged attention, GEMM, graph replay, collectives.
- $\mathcal{M}_d$: allocation, streams, KV/cache ownership.
- $\mathcal{F}_d$: enabled graph, dtype, kernel-library features.
- $\mathcal{N}_d$: tolerance and fallback behavior.

A capability surface answers one scheduler question: can this worker run the selected step without changing what it means?

Capability state is an intersection, not a device name

Definition (Capability state)
$S_d(t) = (F_d^{\text{device}}, F_d^{\text{backend}}, F_t^{\text{runtime}}, R_t^{\text{res}})$. A path p is legal only when $\text{requires}(p) \subseteq F_t^{\text{runtime}}$ and residency R_t^{res} covers its memory.

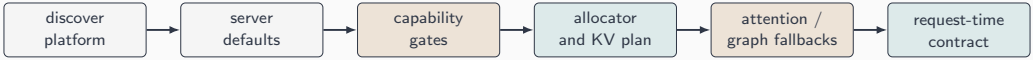
The runtime enables only the intersection:

$$F_{\text{runtime}} \subseteq F_{\text{device}} \cap F_{\text{backend}}.$$

A feature can be physically possible and still unavailable under the current backend, dtype, model family, or memory layout — so graph replay falls back while preserving request ids and cache-slot ownership.

Discovery, memory, and execution

Platform discovery publishes a plan before scheduling



- A ladder, not a backend-name lookup: defaults, gates, owned state, fallbacks, then the contract requests inherit.
- The handoff record $H_d(t) = (\pi, \kappa, P, A, K, Q, G, \Sigma, C, \Gamma, \Phi, T)$ names identity, page size, allocator, KV pool, attention, graph runner, streams, compile, communicator, fallback, tolerance.

$S_d(t)$ says which features are legal; $H_d(t)$ says which objects the next request step consumes.

Memory ownership: same request, different page geometry

A request with **257** retained tokens, two page geometries:

$$\left\lceil \frac{257}{16} \right\rceil = 17 \text{ pages} = 272 \text{ slots} \quad \text{vs.} \quad \left\lceil \frac{257}{128} \right\rceil = 3 \text{ pages} = 384 \text{ slots.}$$

- The scheduler sees *one* request length; the allocator sees a different resource shape (15 vs. 127 slack slots).
- CUDA paged KV, CPU host memory, MLX pool-backed cache, NPU page-first buffers: different slot-map and storage owners, *same* request-to-cache obligation.

Neither geometry is wrong. A portable comparison separates request semantics from page geometry.

Execution modes change when a result becomes visible

Every mode moves the launch or materialization point — but a later consumer must never read unfinished work, stale cache rows, or wrong-shape metadata.

- **CUDA graph replay**: a fixed launch skeleton with stable addresses.
- **MLX lazy overlap**: a deferred handle lets the next pure-decode job queue before the previous tokens are forced.
- **CPU packed kernels**: thread/NUMA binding, host-memory KV, shared-memory collectives.
- **PD-mux split prefill**: decode and a layer-sliced prefill share one GPU on two Green Context streams.

Different completion disciplines, one visibility rule.

Correctness and evidence

The two invariants that gate every backend

Invariant (Backend step-alignment)
Fallbacks may change performance but must not change request semantics, cache ownership, or sampling results beyond documented numerical tolerance.

Invariant (Backend feature coverage)
An account is valid only when feature support, allocator/pool ownership, kernel/graph choices, transfer paths, fallback, tolerance, CI coverage, and documentation describe the same device contract.

Step-alignment protects the request; feature coverage protects the claim. A checkbox is an aspiration until allocator, kernel, tolerance, and test evidence agree.

Evidence class controls the strongest claim

Row	Evidence	Strongest claim
CUDA completed	RTX 5090 / SM120, Qwen3-8B, FlashInfer, graph + overlap, latency, memory	narrow local optimized-path row
CPU diagnostic	tiny model, host KV cap, `torch_native`, TTFT	fallback contract, not a benchmark
MLX diagnostic	Apple-silicon, `MlxKVPool`, lazy path	local diagnostic only
PD-mux blocked	`--enable-pdmux`, stopped on SM120	mechanism + a capability boundary
TPU source-backed	SGLang-JAX, XLA, topology, RPA	compiler-runtime boundary
Source-only	NPU/MUSA/ROCm/XPU surfaces	obligations, no speed claim

The same serving contract appears in every row; the evidence class decides what it may prove.

What to carry forward

1. A backend is a **contract**, not a device name: prove capability before you enable a path.
2. **Discovery** publishes objects the hot path trusts — allocator, KV pool, attention, graph runner, dispatch key.
3. **Page geometry** changes the resource shape while the request-to-cache obligation stays fixed.
4. **Execution modes** move the completion point; a consumer must never read unfinished or stale state.
5. **Two invariants** — step-alignment and feature coverage — and an **evidence class** decide every backend claim.

Next: from the portability contract to platform discovery — how a device string becomes the pools, allocators, and runners a serving step inherits.